

# Distributed Messaging

## Kafka & RabbitMQ — Study Notes

Topics · Partitions · Offsets · Consumer Groups · ZooKeeper  
Exchange Types · Push vs Pull · DLQ · Fault Tolerance

---

### TOPICS COVERED

1. What is a Messaging Queue?

2. P2P vs Publish-Subscribe

3. Kafka Architecture Deep Dive

4. Topics, Partitions, Consumer Groups

5. Kafka Fault Tolerance & Offsets

6. RabbitMQ Architecture

7. Exchange Types (Fanout/Direct/Topic)

8. Kafka vs RabbitMQ Comparison

## Table of Contents

---

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>1. What is a Messaging Queue?</b>                    | <b>3</b> |
| <b>2. Why Messaging Queues?</b>                         | <b>3</b> |
| <b>3. P2P vs Publish-Subscribe</b>                      | <b>3</b> |
| <b>4. Kafka Architecture Deep Dive</b>                  | <b>4</b> |
| <b>5. Topics, Partitions &amp; Consumer Groups</b>      | <b>4</b> |
| <b>6. Kafka Fault Tolerance &amp; Offset Management</b> | <b>5</b> |
| <b>7. RabbitMQ Architecture &amp; Exchange Types</b>    | <b>5</b> |
| <b>8. Push vs Pull — RabbitMQ vs Kafka</b>              | <b>6</b> |
| <b>9. Handling Failures — DLQ &amp; Retries</b>         | <b>6</b> |
| <b>10. Kafka vs RabbitMQ Comparison</b>                 | <b>7</b> |
| <b>12. Quick Revision &amp; Interview Questions</b>     | <b>8</b> |

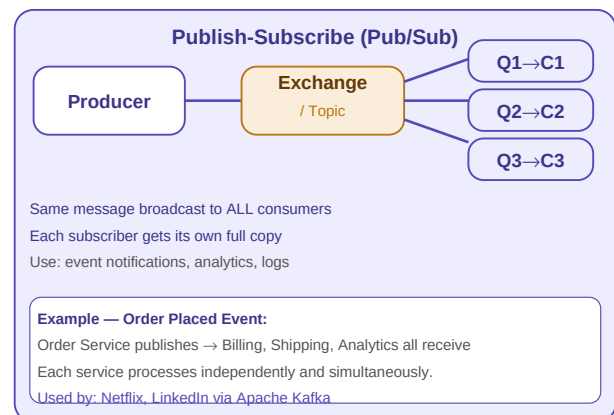
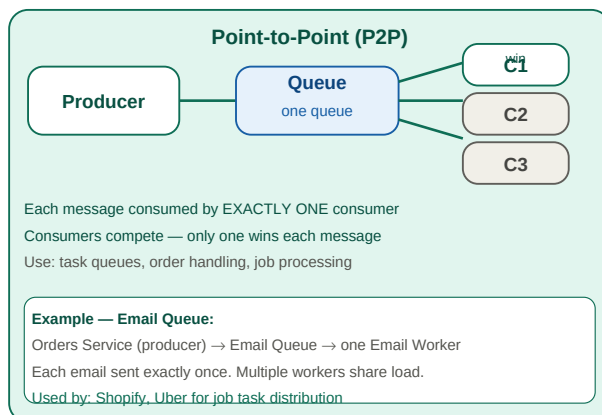
## 1. What is a Messaging Queue & Why Use One?

A **messaging queue** decouples producers (who generate events/tasks) from consumers (who process them). Instead of calling each other directly, they communicate via a shared queue asynchronously. This is fundamental to scalable microservice architectures.

| Advantage        | Without Queue                           | With Queue                                   |
|------------------|-----------------------------------------|----------------------------------------------|
| Coupling         | Producer waits for consumer to finish   | Producer sends and moves on immediately      |
| Failure handling | If consumer fails, request is lost      | Message stays in queue until consumed        |
| Speed mismatch   | Producer slows down if consumer is slow | Queue absorbs the difference (pace matching) |
| Scalability      | Direct calls limit throughput           | Add more consumers to scale processing       |

## 2 & 3. Messaging Patterns — P2P vs Publish-Subscribe

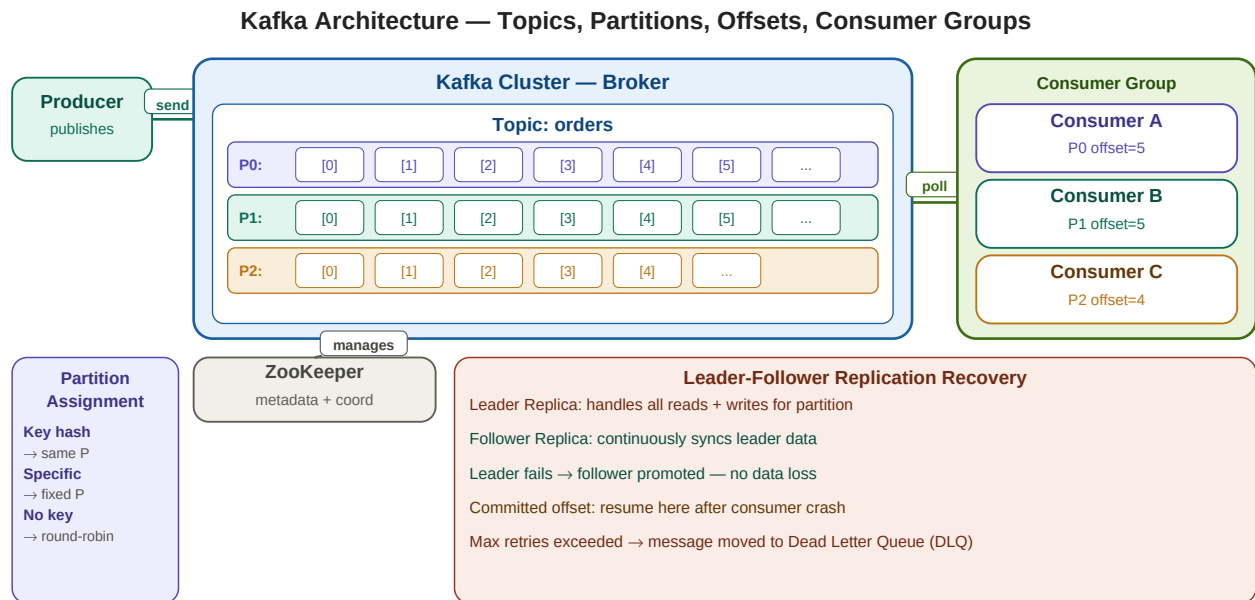
### Messaging Patterns — P2P vs Publish-Subscribe



P2P: message consumed by exactly one consumer · Pub-Sub: message broadcast to all subscribers · each gets their own copy

## 4 & 5. Kafka Architecture — Topics, Partitions, Consumer Groups

**Kafka** is a distributed log-based messaging system. Producers publish messages to **topics**. Topics are split into **partitions** which are stored across brokers. Consumers in a **consumer group** each read from one or more partitions — ensuring parallel processing without duplication.



Kafka: Producer → Broker (Topic/Partitions with offsets) → Consumer Group · Leader-Follower replication · ZooKeeper for metadata

### Partition Assignment Rules

- **Key provided:**  $\text{hash}(\text{key}) \% \text{num\_partitions}$  — same key always goes to same partition (ordering guarantee)
- **Specific partition:** producer explicitly sets partition number
- **No key:** round-robin across all partitions (load distribution)

### Consumer Group Rules

- Each partition assigned to exactly ONE consumer in a group — prevents duplicate processing
- Multiple consumer groups can read the same topic independently (Pub-Sub behaviour)
- If a consumer fails, its partitions are reassigned to other consumers in the group
- Adding more consumers than partitions = idle consumers (no benefit)

## 6. Kafka Fault Tolerance & Offset Management

| Scenario                  | What Happens                                    | Recovery Mechanism                  |
|---------------------------|-------------------------------------------------|-------------------------------------|
| Broker (leader) fails     | Follower replica is promoted to leader          | No data loss — follower was in sync |
| Consumer fails            | Other consumers in group pick up its partitions | Resume from last committed offset   |
| Consumer retries exceeded | Message moved to Dead Letter Queue (DLQ)        | DLQ processed separately / alerting |

| Scenario          | What Happens                                                 | Recovery Mechanism                          |
|-------------------|--------------------------------------------------------------|---------------------------------------------|
| Partition full    | Add more brokers horizontally                                | Partitions redistributed across new brokers |
| Network partition | ISR (in-sync replicas) tracked — only sync replicas promoted | Prevents stale data from becoming leader    |

Committed offset: the last successfully processed message position. Consumer failure → resume from here.

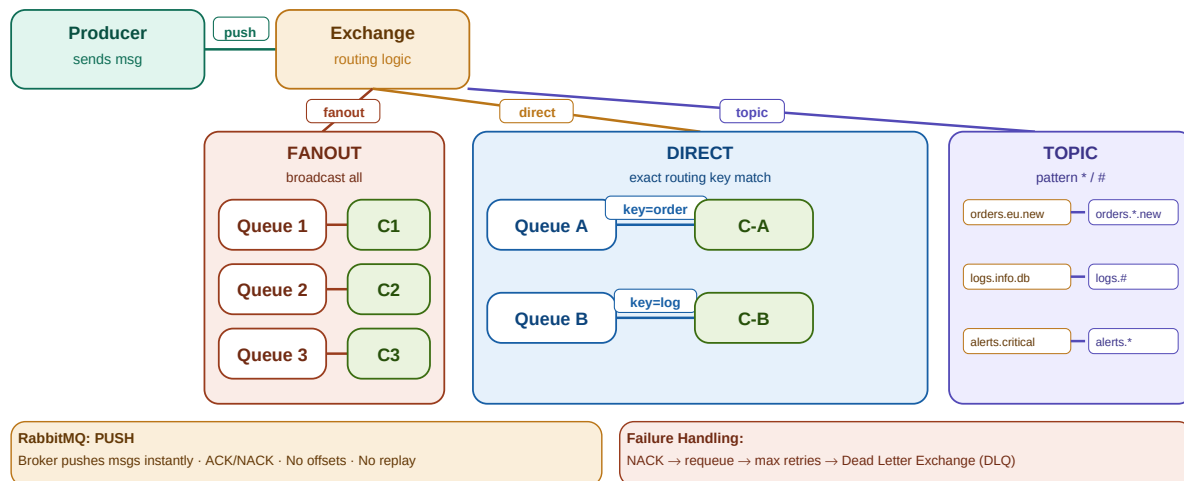
In-Sync Replicas (ISR): only brokers that are fully synced with leader are candidates for promotion.

Dead Letter Queue (DLQ): messages that failed after max retries are moved here for manual inspection.

## 7 & 8. RabbitMQ Architecture — Exchange Types & Push Model

In **RabbitMQ**, producers send messages to an **Exchange** — not directly to a queue. The exchange routes messages to queues based on **routing keys** and **bindings**. Consumers then receive messages from queues. RabbitMQ uses a **push model** — the broker pushes messages to consumers as they arrive.

RabbitMQ Architecture — Exchange Types Push Model



RabbitMQ: Producer → Exchange → Queues → Consumers · Three exchange types: Fanout / Direct / Topic · Push model vs Kafka Pull

### Exchange Types

| Exchange Type | Routing Logic                                            | Use Case                               | Example                                            |
|---------------|----------------------------------------------------------|----------------------------------------|----------------------------------------------------|
| Fanout        | Broadcast to ALL bound queues — ignores routing key      | Event notifications to all subscribers | Order placed → notify billing, shipping, analytics |
| Direct        | Route to queue with EXACT routing key match              | Task routing by type or priority       | routing key=email → Email Queue only               |
| Topic         | Pattern matching on routing key (* = one word, # = many) | Hierarchical routing                   | logs.*.error → all error logs from any service     |

### RabbitMQ vs Kafka — Push vs Pull

| Aspect           | RabbitMQ (Push)                               | Kafka (Pull)                                 |
|------------------|-----------------------------------------------|----------------------------------------------|
| Mechanism        | Broker pushes message to consumer immediately | Consumer polls broker for new messages       |
| Consumer control | Broker decides when consumer gets message     | Consumer controls its own processing rate    |
| Backpressure     | Consumer can NACK to slow down or reject      | Consumer simply stops polling if overwhelmed |
| Offset tracking  | None — relies on ACK/NACK per message         | Yes — offset stored, replayable              |

| Aspect         | RabbitMQ (Push)                 | Kafka (Pull)                  |
|----------------|---------------------------------|-------------------------------|
| Message replay | Not supported — consumed = gone | Yes — seek to any past offset |

## 9. Handling Failures — DLQ & Retries

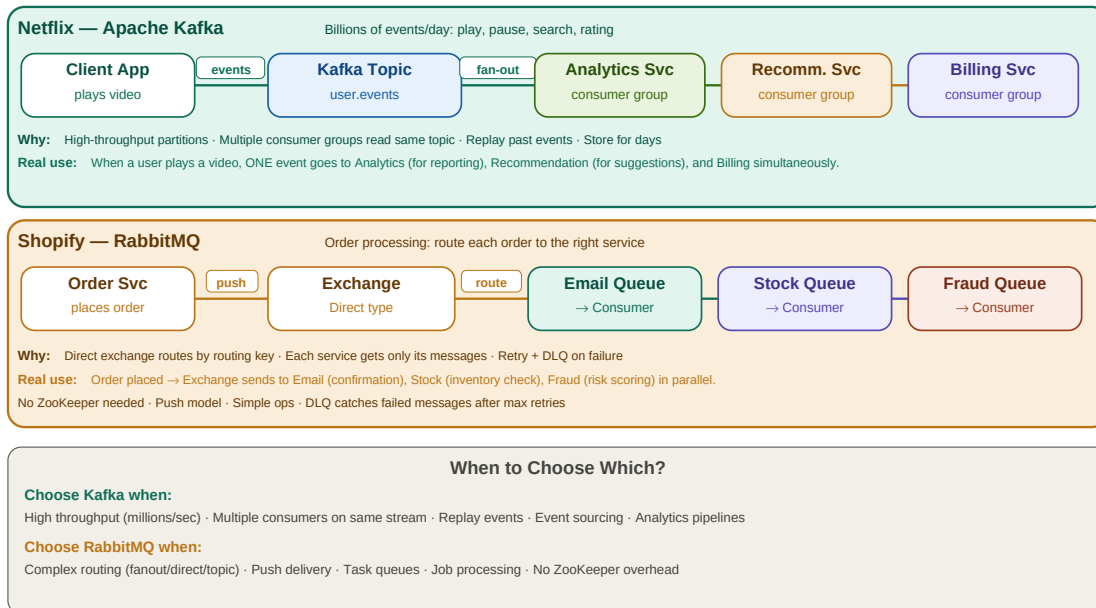
| Scenario                        | Kafka                             | RabbitMQ                               |
|---------------------------------|-----------------------------------|----------------------------------------|
| Consumer crashes mid-processing | Resume from last committed offset | Message re-queued via NACK             |
| Processing fails after retries  | Move to Dead Letter Topic (DLT)   | Move to Dead Letter Exchange (DLX)     |
| Queue/topic full                | Add brokers (horizontal scale)    | Messages dropped or persisted (config) |
| Replay old messages             | Seek to any past offset           | Not possible — consumed messages gone  |
| Multiple consumers same msg     | Use separate consumer groups      | Each queue has its own consumers       |

## 10. Real-World Designs — Netflix & Shopify

**Netflix** processes billions of events per day (play, pause, search, rating) using **Apache Kafka**. Each event published to a topic is independently consumed by analytics, recommendation, and billing services — classic Pub-Sub with high throughput and replay capability.

**Shopify** uses **RabbitMQ** to process orders. An order event is routed via a Direct exchange to separate queues for email confirmation, inventory update, and fraud checking — each running independently with retry logic and DLQ for failed orders.

### Real-World Use Cases — Linear Architecture Flows



Netflix: Kafka for high-throughput event streaming · Shopify: RabbitMQ for complex order routing

## 11. Kafka vs RabbitMQ — Complete Comparison

### Kafka vs RabbitMQ — Complete Comparison

| Aspect           | Kafka                         | RabbitMQ                         |
|------------------|-------------------------------|----------------------------------|
| Delivery model   | Pull — consumer polls broker  | Push — broker pushes to consumer |
| Message ordering | Guaranteed per partition      | Guaranteed per queue             |
| Offset tracking  | Yes — consumer manages        | No — ACK/NACK based              |
| Message routing  | Topic + partition key         | Exchange + routing key           |
| Retention        | Configurable (hours/days)     | Until ACK'd and consumed         |
| Replay messages  | Yes — seek to any past offset | No — consumed = gone             |
| Exchange types   | N/A (topic-based)             | Fanout / Direct / Topic          |
| Scalability      | Horizontal — add brokers      | Clustering + sharding            |
| ZooKeeper needed | Yes (Kafka 2.x) / KRaft (3.x) | No — built-in coordination       |
| Best for         | High-throughput event streams | Complex routing, low latency     |
| Real-world use   | Netflix, LinkedIn, Uber       | Shopify, Instagram, Reddit       |

11 dimensions compared · Kafka: pull, offset-tracked, replayable · RabbitMQ: push, exchange-routed, low-latency



## 12. Quick Revision & Interview Questions

### Quick Revision

- > Messaging queue: decouples producer and consumer — async, retry, pace matching.
- > P2P: one message → one consumer (compete). Pub-Sub: one message → all subscribers.
- > Kafka: topics divided into partitions. Messages stored sequentially with offsets.
- > Offset: sequential ID of a message within a partition. Tracks consumer progress.
- > Consumer group: partitions split among consumers — each partition to exactly one consumer.
- > Multiple consumer groups can read same topic independently (Pub-Sub behaviour).
- > Partition assignment: key hash → same partition; no key → round-robin.
- > Leader replica: handles reads/writes. Follower: replicates. On failure, follower promoted.
- > Committed offset: last successful read. Consumer failure → resume from here.
- > DLQ: dead letter queue — messages moved here after max retries exceeded.
- > Kafka: PULL model — consumer polls. RabbitMQ: PUSH model — broker pushes.
- > ZooKeeper: manages Kafka broker metadata, topic/partition assignments.
- > RabbitMQ exchange types: Fanout (broadcast), Direct (exact key), Topic (pattern).
- > Kafka: replay messages by seeking back. RabbitMQ: consumed = gone, no replay.
- > Kafka: best for high-throughput event streams, log aggregation, analytics.
- > RabbitMQ: best for complex routing, low-latency task queues, notifications.

### Interview Questions

- 1. What is a messaging queue? Why is it used in microservices?
- 2. Explain P2P vs Publish-Subscribe messaging patterns.
- 3. Walk me through Kafka's architecture end-to-end.
- 4. What is a Kafka partition and why does it matter?
- 5. How does a Kafka consumer group work?
- 6. What is an offset in Kafka? How is it used for fault recovery?
- 7. What is a Dead Letter Queue? When is a message sent there?
- 8. Explain the leader-follower replication model in Kafka.
- 9. What is RabbitMQ's exchange? What are the three exchange types?
- 10. What is the difference between push and pull messaging models?
- 11. When would you choose Kafka over RabbitMQ? And vice versa?
- 12. What is ZooKeeper's role in Kafka?
- 13. How does Kafka handle a consumer that crashes mid-processing?
- 14. Can two consumer groups read the same Kafka topic? What is the result?

### Key Terms

|                       |                                                                         |
|-----------------------|-------------------------------------------------------------------------|
| <b>Topic</b>          | Named logical channel in Kafka — producers publish to topics            |
| <b>Partition</b>      | Subdivision of a topic — enables parallelism and ordering per partition |
| <b>Offset</b>         | Sequential message ID within a partition — tracks consumer position     |
| <b>Consumer Group</b> | Consumers sharing workload — each partition read by only one member     |

|                            |                                                                           |
|----------------------------|---------------------------------------------------------------------------|
| <b>Broker</b>              | Kafka server storing topics and partitions                                |
| <b>ZooKeeper</b>           | Manages Kafka cluster metadata: broker info, partition leaders            |
| <b>Leader Replica</b>      | Primary copy of a partition — handles all reads and writes                |
| <b>Follower Replica</b>    | Sync copy of leader — promoted on leader failure                          |
| <b>Committed Offset</b>    | Last successfully processed message — recovery resumes here               |
| <b>DLQ / DLT</b>           | Dead Letter Queue/Topic — receives messages that failed after retries     |
| <b>Exchange (RabbitMQ)</b> | Routing middleware — Fanout/Direct/Topic determines queue assignment      |
| <b>Routing Key</b>         | RabbitMQ message label used by Exchange to route to correct queue         |
| <b>Fanout</b>              | Exchange type: broadcast to ALL bound queues                              |
| <b>Direct</b>              | Exchange type: route to queue with exact routing key match                |
| <b>Topic Exchange</b>      | Exchange type: pattern match routing key (* = 1 word, # = many)           |
| <b>Push model</b>          | Broker actively sends messages to consumer (RabbitMQ)                     |
| <b>Pull model</b>          | Consumer polls broker for new messages (Kafka)                            |
| <b>ACK / NACK</b>          | RabbitMQ: acknowledge (success) or negative-acknowledge (failure/requeue) |